PANDUAN OPERASIONAL LARAVEL WORKER, SCHEDULER & CRONJOB

Arsitektur Background Processing, Antrean Tugas, dan Otomatisasi Terjadwal Livirooms

Workspace Sumber: `laravel/` (Laravel 12 & PostgreSQL)
Target Pembaca: DevOps Engineer, System Administrator, Backend Developer, Technical Lead
Versi Dokumen: v1.0.0 (Production Ready)

DAFTAR ISI

1. Bab 1: Konsep Dasar & Arsitektur Background Processing
   - 1.1 Perbedaan Scheduler, Queue Worker, Job Dispatch, dan Cronjob
   - 1.2 Aliran Data Pemrosesan Asinkron (Async Lifecycle)
   - 1.3 Driver Antrean Database (`QUEUE_CONNECTION=database`)
2. Bab 2: Inventaris Job, Command & Jadwal di Livirooms
   - 2.1 Ringkasan Seluruh Komponen Terjadwal & Asinkron
   - 2.2 Sweep Pembayaran Kedaluwarsa (`payments:cancel-expired`)
   - 2.3 Auto-Cancel Booking Hotel & Restorasi Kamar (`CancelExpiredBookingsJob`)
   - 2.4 Ekstensi Inventaris Kamar Harian (`ExtendHotelInventoriesJob`)
   - 2.5 Penerbitan Polis Asuransi Tamu (`IssueInsurancePolicyJob`)
   - 2.6 Sinkronisasi Status Polis Asuransi Berkala (`SyncInsuranceStatusJob`)
3. Bab 3: Mekanisme Penulisan & Pemanggilan Job (Dispatching)
   - 3.1 Sintaks Pemanggilan (`dispatch` , `dispatchSync` , `dispatchAfterResponse`)
   - 3.2 Penundaan Waktu Eksekusi (*Delayed Dispatch*)
   - 3.3 Penanganan Percobaan Ulang (*Retries, Backoff & Timeout*)
   - 3.4 Penanganan Kegagalan (*Failed Job Handling*)
4. Bab 4: Panduan Setup di VPS Linux (Ubuntu / Debian / AlmaLinux)
   - 4.1 Menyiapkan Crontab Linux untuk Laravel Scheduler
   - 4.2 Instalasi & Konfigurasi Supervisord untuk Queue Worker
   - 4.3 Menjalankan Worker dengan Concurrency (Multi-Process)
   - 4.4 Manajemen Layanan Supervisor (`supervisorctl`)
   - 4.5 Alur Deployment & Perintah Wajib `queue:restart`

5. Bab 5: Panduan Setup di Lingkungan cPanel (Shared Hosting / Cloud)
    - 5.1 Menemukan Jalur Eksekusi PHP Binary Path yang Tepat
    - 5.2 Mengatur Cron Job Scheduler di cPanel
    - 5.3 Menjalankan Queue Worker di cPanel via Cron `--stop-when-empty`
    - 5.4 Alternatif Eksekusi Worker via SSH / Terminal cPanel
    - 5.5 Penyesuaian `memory_limit` dan `max_execution_time`
6. Bab 6: Pemantauan, Diagnosa & Penanganan Masalah (Troubleshooting)
    - 6.1 Memantau Antrean Aktif & Pekerjaan yang Gagal (`failed_jobs`)
    - 6.2 Perintah Diagnosa Artisan (`queue:monitor` , `queue:retry` , `queue:failed`)
    - 6.3 Menelaah Log Error di `storage/logs/laravel.log`
    - 6.4 Lembar Ceklis Kesiapan Produksi (*Production Checklist*)

BAB 1: KONSEP DASAR & ARSITEKTUR BACKGROUND PROCESSING

1.1 Perbedaan Scheduler, Queue Worker, Job Dispatch, dan Cronjob

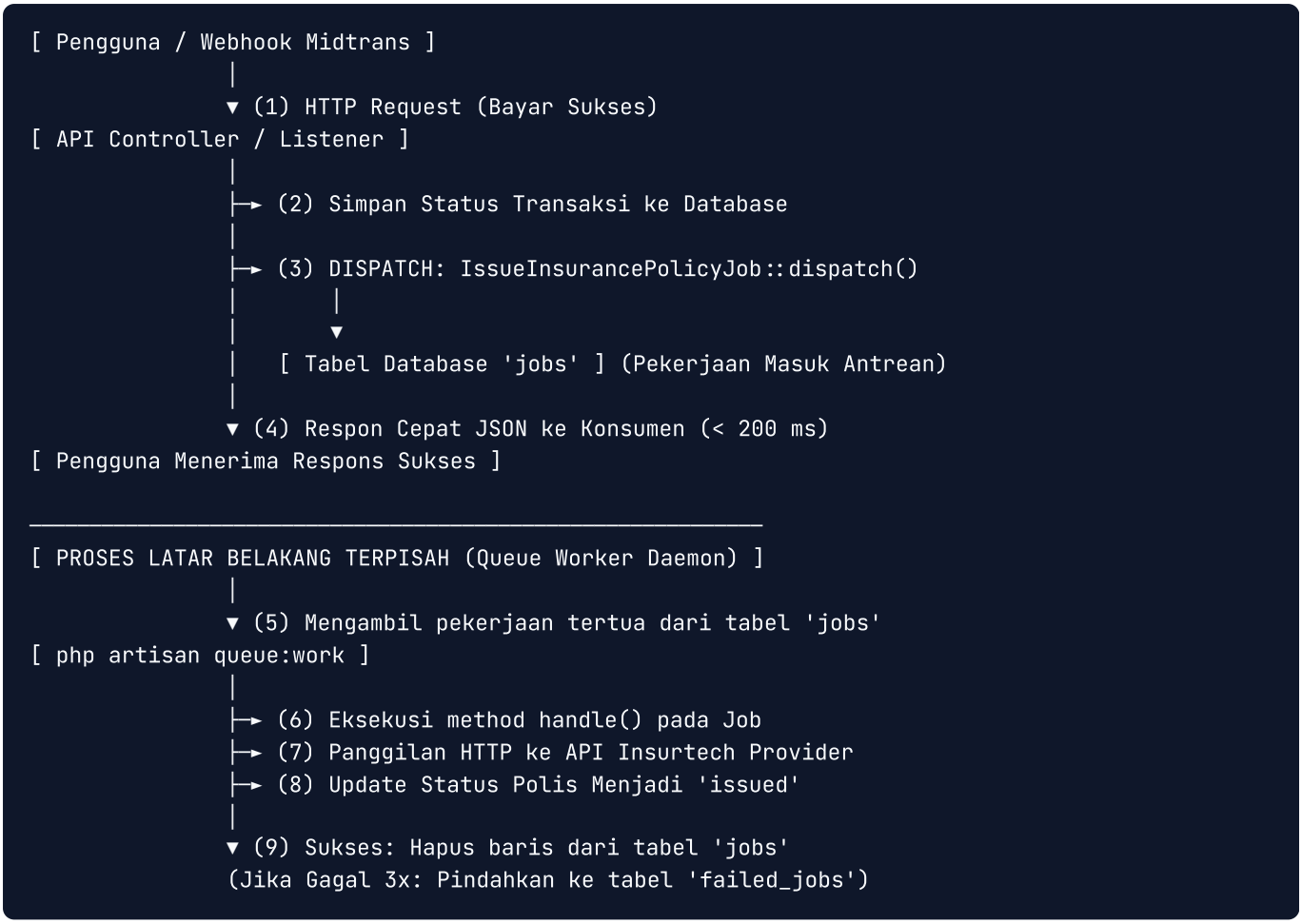
Dalam arsitektur backend Livirooms berbasis **Laravel 12**, pemrosesan tugas dibagi menjadi dua domain besar: **Siklus Request-Response HTTP (Sinkron)** dan **Pemrosesan di Belakang Layar (Asinkron)**.

Empat istilah kunci yang saling berkolaborasi:

KOMPONEN	DEFINISI & PERAN	CONTOH DI LIVIROOMS
Cronjob	Mekanisme penjadwalan bawaan sistem operasi (Linux/Unix daemon) yang mengeksekusi perintah pada interval waktu tertentu.	Mengeksekusi `php artisan schedule:run` setiap 1 menit dari server.
Scheduler (Laravel Scheduler)	Manajer jadwal internal aplikasi di mana seluruh jadwal tugas didefinisikan secara ekspresif dalam kode PHP tanpa mengotak-atik crontab server berulang kali.	Mendefinisikan perintah `payments:cancel-expired` berjalan setiap 10 menit.
Job (Queued Job)	Kelas terisolasi yang mewakili sebuah tugas spesifik yang dapat dieksekusi di luar alur utama pengguna untuk mencegah request timeout.	`IssueInsurancePolicyJob` yang memanggil API pihak ketiga untuk menerbitkan polis.
Dispatch	Perintah untuk mendorong (*push*) sebuah Job ke dalam antrean (*Queue*) atau mengeksekusinya secara langsung.	`IssueInsurancePolicyJob::dispatch($insuranceId);` saat pembayaran lunas.
Queue Worker	Proses jangka panjang (*long-running daemon*) yang terus-menerus memantau antrean dan mengeksekusi Job yang masuk satu per satu.	`php artisan queue:work --tries=3` yang dipantau oleh Supervisord.

1.2 Aliran Data Pemrosesan Asinkron (Async Lifecycle)

Diagram alur berikut menggambarkan bagaimana permintaan pengguna dialihkan ke background worker:



1.3 Driver Antrean Database (`QUEUE_CONNECTION=database`)

Di Livirooms, driver antrean default menggunakan **PostgreSQL Database** (`QUEUE_CONNECTION=database` pada berkas `.env`).

Struktur tabel pendukung di basis data:

1. **`jobs`** : Menyimpan seluruh pekerjaan yang sedang menunggu (*pending*) atau sedang diproses (*reserved*) oleh worker. Baris akan otomatis terhapus saat pekerjaan selesai dengan sukses.
2. **`failed_jobs`** : Menyimpan pekerjaan yang gagal dieksekusi setelah melampaui batas percobaan maksimal (`$tries = 3`). Menyimpan exception stack trace untuk audit.
3. **`job_batches`** : Menyimpan status batch jika beberapa job dieksekusi secara berbarengan.

> ⚠️ **CATATAN KRUSIAL:**
> Jika `QUEUE_CONNECTION=sync` digunakan di `.env` , seluruh Job akan dijalankan secara sinkron (blocking). Akibatnya, jika API pihak ketiga lambat, aplikasi mobile atau browser pengguna akan mengalami *freezing* atau *504 Gateway Timeout*. Pastikan lingkungan produksi selalu menggunakan `QUEUE_CONNECTION=database` atau `redis` .

BAB 2: INVENTARIS JOB, COMMAND & JADWAL DI LIVIROOMS

Seluruh jadwal otomatis Livirooms terdaftar secara terpusat pada berkas `routes/console.php`.

2.1 Ringkasan Seluruh Komponen Terjadwal & Asinkron

NAMA KELAS / PERINTAH	TIPE	PEMICU / FREKUENSI	TUJUAN & LOGIKA BISNIS
`payments:cancel-expired`	Artisan Command	Scheduler (`everyTenMinutes()`)	Memeriksa status Midtrans untuk pembayaran tertunda yang melebihi batas waktu Snap, membatalkannya, dan membatalkan pesanan yang tanggal check-in-nya sudah terlewat tanpa pembayaran.
`CancelExpiredBookingsJob`	Queued Job	Scheduler (`everyFiveMinutes()`)	Membatalkan reservasi hotel (`booking_type='room'`) yang belum dibayar dalam 30 menit, serta **mengembalikan stok kamar fisik** (`unit_types.remaining`).
`ExtendHotelInventoriesJob`	Queued Job	Scheduler (`dailyAt('00:00')`)	Pemeliharaan kalender inventaris harian. Saat ini berfungsi sebagai no-op aman pasca refactoring konsolidasi counter stok kamar.
`SyncInsuranceStatusJob`	Queued Job	Scheduler (`hourly()`)	Sinkronisasi status klaim/penerbitan asuransi ke API mitra asuransi untuk polis dalam 30 hari terakhir.
`IssueInsurancePolicyJob`	Queued Job	Event Listener (`BookingPaid`) / Admin Action	Memanggil HTTP API Insurtech Provider untuk mencetak nomor polis resmi asuransi perjalanan tamu secara asinkron. Dilengkapi retry 3× dan backoff 30 detik.

2.2 Sweep Pembayaran Kedaluwarsa (`payments:cancel-expired`)

- **Berkas Kode:** `app/Console/Commands/CancelExpiredPayments.php`
- **Definisi Jadwal:**

```
Schedule::command('payments:cancel-expired')
    ->everyTenMinutes()
    ->withoutOverlapping()
    ->onFailure(function (\Throwable $e) {
        Log::error('Failed to run payments:cancel-expired', ['error' => $e->getMessage()]);
    });
```

- **Cara Kerja & Keamanan:** Perintah ini tidak langsung membatalkan pembayaran berdasarkan umur transaksi semata. Perintah ini **selalu menanyakan status faktual ke Midtrans API Gateway terlebih dahulu**. Hal ini mencegah pembatalan salah sasaran jika webhook dari Midtrans sempat terputus sementara (*network drop*).

2.3 Auto-Cancel Booking Hotel & Restorasi Kamar (`CancelExpiredBookingsJob`)

- **Berkas Kode:** `app/Jobs/CancelExpiredBookingsJob.php`
- **Batas Waktu (*Cutoff*):** 30 Menit (`HOTEL_EXPIRY_MINUTES = 30`).
- **Logika Restorasi Inventaris:**

```
foreach ($booking->bookingRooms as $br) {
    $unitType = UnitType::find($br->room_type_id);
    if ($unitType && $unitType->remaining !== null) {
        $unitType->increment('remaining', $br->quantity);
    }
}
```

  Jika tamu memilih kamar namun tidak menyelesaikan transaksi dalam 30 menit, status pesanan diubah ke `expired` , asuransi dibatalkan, dan kuota kamar (`remaining`) segera dikembalikan agar dapat disewa oleh tamu lain.

2.4 Penerbitan Polis Asuransi Tamu (`IssueInsurancePolicyJob`)

- **Berkas Kode:** `app/Jobs/IssueInsurancePolicyJob.php`
- **Konfigurasi Percobaan Ulang:**

```
public int $tries = 3;
public int $backoff = 30; // Jeda 30 detik sebelum mencoba lagi
```

- **Parameter Payload API Provider:** Mengirimkan kode booking mitra, kode paket asuransi, identitas pemegang polis, nomor KTP, tanggal check-in/check-out, jumlah tamu, nilai premi, dan nama hotel. Jika API mitra mengalami gangguan (500/503), worker akan otomatis mencoba kembali sebanyak 3 kali dengan jeda 30 detik.

2.5 Sinkronisasi Status Polis Asuransi Berkala (`SyncInsuranceStatusJob`)

- **Berkas Kode:** `app/Jobs/SyncInsuranceStatusJob.php`
- **Frekuensi:** Dijalankan setiap 1 jam via Scheduler.
- **Tujuan:** Melakukan penarikan data (*polling*) ke endpoint provider `/v1/policies/{policy_number}/status` untuk memetakan perubahan status seperti klaim cair (*claimed*) atau pembatalan perlindungan (*cancelled*).

BAB 3: MEKANISME PENULISAN & PEMANGGILAN JOB (DISPATCHING)

3.1 Sintaks Pemanggilan (`dispatch` , `dispatchSync` , `dispatchAfterResponse`)

Di dalam Controller, Service, atau Event Listener, Job dapat dipanggil dengan berbagai variasi:

```
use App\Jobs\IssueInsurancePolicyJob;

// 1. Asinkron standar: Masuk ke antrean database (Recommended)
IssueInsurancePolicyJob::dispatch($insuranceId);

// 2. Sinkron: Langsung dieksekusi saat itu juga di thread request saat ini
IssueInsurancePolicyJob::dispatchSync($insuranceId);

// 3. Pasca-Respons: Dijalankan langsung setelah HTTP response dikirim ke browser/client
IssueInsurancePolicyJob::dispatchAfterResponse($insuranceId);
```

3.2 Penundaan Waktu Eksekusi (*Delayed Dispatch*)

Jika Job perlu dieksekusi beberapa waktu setelah suatu aksi:

```
// Menunda eksekusi selama 15 menit ke depan
IssueInsurancePolicyJob::dispatch($insuranceId)->delay(now()->addMinutes(15));
```

3.3 Penanganan Percobaan Ulang (*Retries, Backoff & Timeout*)

Properti standar yang wajib ditentukan di dalam kelas Job untuk stabilitas koneksi jaringan eksternal:

```
class MyExternalApiJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    /** Jumlah maksimal percobaan sebelum dinyatakan gagal permanen */
    public int $tries = 3;

    /** Jeda waktu antar percobaan (detik) */
    public int $backoff = 30;

    /** Batas maksimal durasi eksekusi per percobaan (detik) */
    public int $timeout = 60;
}
```

BAB 4: PANDUAN SETUP DI VPS LINUX (UBUNTU / DEBIAN / ALMALINUX)

Pada lingkungan Virtual Private Server (VPS) mandiri, dua komponen wajib diaktifkan secara terpisah:

1. **Linux Crontab:** Untuk memicu `php artisan schedule:run` setiap menit.
2. **Supervisord Daemon:** Untuk menjaga `php artisan queue:work` tetap aktif 24/7 dan me-restart secara otomatis jika crash.

4.1 Menyiapkan Crontab Linux untuk Laravel Scheduler

Masuk ke server via SSH sebagai root atau user dengan hak akses sudo:

1. Buka konfigurasi crontab untuk pengguna web server (misalnya `www-data` di Ubuntu atau `nginx` di CentOS/AlmaLinux):

   ```
   sudo crontab -e -u www-data
   ```

2. Tambahkan satu baris berikut di baris paling bawah:

   ```
   * * * * * cd /var/www/livirooms/laravel && php artisan schedule:run >> /dev/null 2>&1
   ```

   (Sesuaikan `/var/www/livirooms/laravel` dengan path direktori proyek Anda).

3. Simpan dan keluar dari editor (`Ctrl + O` , `Enter` , `Ctrl + X` pada nano).

4. Verifikasi jadwal crontab:

   ```
   sudo crontab -l -u www-data
   ```

Setiap menit, cron Linux akan membangunkan Scheduler Laravel. Scheduler akan mengecek berkas `routes/console.php` dan hanya menjalankan perintah yang telah jatuh tempo (setiap 5 menit, 10 menit, atau harian).

4.2 Instalasi & Konfigurasi Supervisord untuk Queue Worker

Supervisord adalah manajer proses untuk sistem Linux yang memantau dan mengontrol proses seperti daemon queue worker.

1. **Instalasi Supervisor:**
   - Di Ubuntu / Debian:

     ```
     sudo apt update && sudo apt install -y supervisor
     ```

   - Di RHEL / AlmaLinux / Rocky Linux:

     ```
     sudo dnf install -y epel-release
     sudo dnf install -y supervisor
     sudo systemctl enable --now supervisord
     ```

2. **Buat File Konfigurasi Worker Livirooms:** Buat file baru di `/etc/supervisor/conf.d/livirooms-worker.conf`:

```
sudo nano /etc/supervisor/conf.d/livirooms-worker.conf
```

3. **Isi Konfigurasi Produksi Terbaik:**

```
[program:livirooms-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/livirooms/laravel/artisan queue:work database --sleep=3 --tries=3 --max-
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=2
redirect_stderr=true
stdout_logfile=/var/www/livirooms/laravel/storage/logs/worker.log
stopwaitsecs=3600
```

Penjelasan Parameter Konfigurasi:

- `command` : Menjalankan worker dengan driver `database`.
  - `--sleep=3` : Jika antrean kosong, jeda 3 detik sebelum query database lagi (menghemat CPU).
  - `--tries=3` : Coba ulang maksimal 3 kali jika ada exception sebelum dipindahkan ke `failed_jobs`.
  - `--max-time=3600` : Me-restart worker setiap 1 jam untuk mencegah kebocoran memori (*memory leak*).
  - `--memory=512` : Batas penggunaan memori per worker (512 MB).
- `numprocs=2` : Menjalankan 2 worker secara paralel (*concurrency*) sehingga antrean tidak menumpuk.
- `user=www-data` : Dijalankan di bawah user web server agar file log dapat ditulis.
- `stopasgroup=true` & `killasgroup=true` : Memastikan seluruh subproses PHP ikut ditutup saat worker dimatikan.

4.3 Manajemen Layanan Supervisor (`supervisorctl`)

Setelah menyimpan berkas konfigurasi, daftarkan dan jalankan worker:

```
# 1. Baca ulang direktori konfigurasi supervisor
sudo supervisorctl reread

# 2. Perbarui proses yang terdaftar
sudo supervisorctl update

# 3. Cek status worker
sudo supervisorctl status
```

Output yang diharapkan:

```
livirooms-worker:livirooms-worker_00   RUNNING   pid 14205, uptime 0:02:10
livirooms-worker:livirooms-worker_01   RUNNING   pid 14206, uptime 0:02:10
```

Perintah operasional penting lainnya:

```
# Menghentikan worker
sudo supervisorctl stop livirooms-worker:*

# Menyalakan worker kembali
sudo supervisorctl start livirooms-worker:*

# Restart seluruh worker
sudo supervisorctl restart livirooms-worker:*
```

4.4 Alur Deployment & Perintah Wajib `queue:restart`

> 🔴 **ATURAN WAJIB DEPLOYMENT:**
> Karena `queue:work` adalah proses daemon yang berjalan terus menerus di memori RAM, worker **tidak akan otomatis membaca perubahan kode PHP baru** setelah Anda melakukan `git pull`. Anda **WAJIB** mengeksekusi perintah `php artisan queue:restart` di akhir proses deploy!

Contoh skrip deployment lengkap (`deploy.sh`):

```
#!/bin/bash
set -e

cd /var/www/livirooms/laravel

echo "1. Mengambil kode terbaru..."
git pull origin main

echo "2. Memperbarui dependensi..."
composer install --no-dev --optimize-autoloader

echo "3. Menjalankan migrasi basis data..."
php artisan migrate --force

echo "4. Mengoptimalkan cache..."
php artisan config:cache
php artisan route:cache
php artisan view:cache

echo "5. Merefresh daemon antrean (Graceful Restart)..."
php artisan queue:restart

echo "Deployment berhasil selesai!"
```

Saat `php artisan queue:restart` dipanggil, worker akan menyelesaikan pekerjaan yang sedang berjalan saat itu secara aman (*gracefully*), lalu berhenti. Supervisord akan langsung menyalakan proses worker yang baru dengan kode PHP terbaru.

BAB 5: PANDUAN SETUP DI LINGKUNGAN CPANEL (SHARED HOSTING / CLOUD)

Pada lingkungan cPanel atau shared hosting, Anda umumnya tidak memiliki akses root dan tidak dapat menginstal Supervisord. Namun, Anda tetap dapat menjalankan Scheduler dan Queue Worker secara andal dengan memanfaatkan menu **Cron Jobs** di cPanel.

5.1 Menemukan Jalur Eksekusi PHP Binary Path yang Tepat

Di cPanel, perintah `php` saja sering kali mengarah ke versi PHP default server yang usang (misal PHP 7.4). Laravel 12 **membutuhkan PHP 8.2 atau PHP 8.3+**.

1. Masuk ke **cPanel** > buka menu **Terminal** (atau cek via SSH).
2. Temukan lokasi pasti binary PHP 8.2/8.3 Anda:

```
which ea-php83 || which ea-php82 || which php
```

3. Contoh path umum di cPanel:
   - `/usr/local/bin/ea-php83` (cPanel EasyApache 4)
   - `/usr/local/bin/ea-php82`
   - `/opt/cpanel/ea-php83/root/usr/bin/php`
   - `/usr/bin/php` (CloudLinux / CageFS)

Catat path ini (contoh kita gunakan: `/usr/local/bin/ea-php83`).

5.2 Mengatur Cron Job Scheduler di cPanel

1. Buka dasbor **cPanel** > cari dan klik menu **Cron Jobs** (Tugas Cron).
2. Di bagian **Add New Cron Job**:
   - **Common Settings:** Pilih **Once Per Minute (`* * * * *`)**.
   - **Minute:** `*`
   - **Hour:** `*`
   - **Day:** `*`
   - **Month:** `*`
   - **Weekday:** `*`

3. Pada kolom **Command**, masukkan perintah:

```
/usr/local/bin/ea-php83 /home/USERNAME/public_html/artisan schedule:run >> /dev/null 2>&1
```

(Ganti `USERNAME` dengan nama akun cPanel Anda, dan sesuaikan folder aplikasi Laravel Anda).

4. Klik tombol **Add New Cron Job**.

Sekarang, Laravel Scheduler sudah aktif dan akan memeriksa tugas berkala setiap menit.

5.3 Menjalankan Queue Worker di cPanel via Cron `--stop-when-empty`

Karena cPanel shared hosting akan membunuh proses background yang berjalan terlalu lama (*long-running process killer*), **JANGAN** menjalankan `artisan queue:work` tanpa parameter penutup!

Solusi Resmi di Shared Hosting:
Gunakan opsi `--stop-when-empty`. Worker akan memproses seluruh pekerjaan yang ada di antrean hingga selesai, lalu keluar secara normal. Cron job akan menjalankannya kembali pada menit berikutnya.

1. Buka kembali menu **Cron Jobs** di cPanel.
2. Atur interval: **Setiap 5 Menit** (`*/5 * * * *`) atau **Setiap Menit** (`* * * * *`).
3. Pada kolom **Command**, masukkan:

```
/usr/local/bin/ea-php83 /home/USERNAME/public_html/artisan queue:work database --stop-when-em
```

4. Klik **Add New Cron Job**.

Mengapa Kombinasi Ini Aman?

- `--stop-when-empty` : Begitu tabel `jobs` kosong, proses PHP selesai dan melepaskan memori.
- `--max-time=50` : Jika ada antrean sangat panjang, worker akan berhenti di detik ke-50 sebelum cron menit berikutnya dijalankan (mencegah tumpang tindih proses).

5.4 Alternatif Eksekusi Worker via SSH / Terminal cPanel

Jika Anda memiliki akses SSH ke cPanel dan hosting mengizinkan proses latar belakang:

1. Anda dapat menjalankan worker dalam sesi terminal persisten menggunakan `nohup` :

```
nohup /usr/local/bin/ea-php83 /home/USERNAME/public_html/artisan queue:work database --sleep=
```

2. Untuk memeriksa apakah worker masih hidup:

```
ps aux | grep artisan
```

5.5 Penyesuaian `memory_limit` dan `max_execution_time`

Jika job Anda melibatkan pembuatan PDF invoice berukuran besar atau panggilan API yang lambat, pastikan konfigurasi PHP di cPanel mencukupi:

1. Buka cPanel > **Select PHP Version** > tab **Options**.
2. Sesuaikan nilai:
   - `memory_limit` : Minimal `512M`
   - `max_execution_time` : Minimal `300` detik
   - `upload_max_filesize` & `post_max_size` : Minimal `20M`

BAB 6: PEMANTAUAN, DIAGNOSA & PENANGANAN MASALAH (TROUBLESHOOTING)

6.1 Memantau Antrean Aktif & Pekerjaan yang Gagal (`failed_jobs`)

Jika sebuah Job gagal dieksekusi 3 kali berturut-turut, Laravel akan memindahkannya dari tabel `jobs` ke tabel `failed_jobs` .

Gunakan perintah Artisan berikut melalui Terminal/SSH:

```
# 1. Melihat daftar pekerjaan yang gagal
php artisan queue:failed

# 2. Mencoba kembali pekerjaan tertentu berdasarkan ID
php artisan queue:retry <ID_JOB>

# 3. Mencoba kembali SELURUH pekerjaan yang gagal
php artisan queue:retry all

# 4. Menghapus pekerjaan yang gagal dari tabel
php artisan queue:forget <ID_JOB>

# 5. Membersihkan seluruh histori failed jobs
php artisan queue:flush
```

6.2 Perintah Diagnosa Artisan Lainnya

- **Memonitor Beban Antrean (Queue Monitor):**

  ```
  php artisan queue:monitor database:default --max=100
  ```

  Menampilkan jumlah pekerjaan yang saat ini sedang mengantre di antrean `default` .

- **Menguji Eksekusi Scheduler Secara Manual:**

  ```
  php artisan schedule:run
  ```

  Mengeksekusi scheduler secara instan tanpa menunggu jam cron berikutnya.

- **Melihat Daftar Tugas Terjadwal Beserta Waktu Eksekusi Berikutnya:**

  ```
  php artisan schedule:list
  ```

  Output menampilkan tabel perintah, interval, dan jadwal eksekusi berikutnya.

6.3 Menelaah Log Error di `storage/logs/laravel.log`

Seluruh kegagalan eksekusi Job dan Scheduler otomatis dicatat oleh Laravel. Jalankan perintah berikut untuk melihat log secara *real-time*:

```
# Memantau error log secara langsung (live tail)
tail -f storage/logs/laravel.log
```

Pola pencarian error penting:

- `CancelExpiredBookingsJob failed`
- `IssueInsurancePolicyJob: insurance_id=... has no booking`
- `SyncInsuranceStatusJob: Provider ... has no api_endpoint`

6.4 Lembar Ceklis Kesiapan Produksi (*Production Checklist*)

Sebelum meluncurkan aplikasi ke lingkungan produksi, pastikan seluruh poin di bawah ini tercentang:

- [ ] `.env` memiliki baris `QUEUE_CONNECTION=database` (bukan `sync`).
- [ ] Tabel database `jobs`, `failed_jobs`, dan `job_batches` sudah ada via `php artisan migrate`.
- [ ] Crontab Linux / cPanel aktif menjalankan `* * * * * ... artisan schedule:run`.
- [ ] Supervisord (VPS) aktif dengan status `RUNNING` untuk `livirooms-worker` (atau cron `--stop-when-empty` di cPanel).
- [ ] File log `/storage/logs/worker.log` dan `/storage/logs/laravel.log` dapat ditulis (*writable* oleh user `www-data`).
- [ ] Skrip deployment menyertakan `php artisan queue:restart`.
- [ ] Konfigurasi kredensial API eksternal (Midtrans & Asuransi) telah diuji coba konektivitasnya.

© 2026 PT Livirooms Indonesia Hospitality. Seluruh Hak Cipta Dilindungi Undang-Undang.